

TP N° 2

Deadline pou la remise du rapport : *Le Samedi 30 Avril avant 20h00.*

Programmation de quantum de temps à l'aide de l'interruption périodique 1CH

Présentation :

Deux méthodes sont utilisées pour implémenter un quantum de temps :

1. Par programmation, en utilisant un compteur avec une boucle dans le programme en cours d'exécution, ceci sera réalisé par le processeur lui-même au détriment du temps de traitement
2. En utilisant un outil externe au microprocesseur afin de décharger ce dernier de la tâche de gestion du temps

Pour le premier cas, une simple boucle ou deux boucles imbriquées sont souvent utilisées pour réaliser des délais de temps. La méthode est très simple à concevoir mais sera réalisée au détriment du temps de traitement (temps processeur).

- exemple d'une seule boucle :

```
                mov cx,0ffffh  
encore :        loop encore
```

- exemple de deux boucles imbriquées :

```
                mov cx, 0ffffh  
boucle_externe : mov si, 0ffffh  
boucle_interne : dec si  
                jnz boucle_interne  
                loop boucle_externe
```

La deuxième méthode étant la plus performante du fait que le délai réalisé sera pris en charge par un outil externe au microprocesseur.

Dans les micro-ordinateurs et indépendamment du système d'exploitation installé, le circuit appelé temporisateur (TIMER) provoque la requête la plus prioritaire dans la machine ayant le numéro 08h sur la broche IRQ0 du contrôleur d'interruptions maître et ce toutes les 18,2 fois/sec, donc avec une périodicité = 55 msec. La routine d'interruption N° 08h appelle elle-même l'interruption logicielle 1CH avec la même périodicité. Cette dernière peut être déroutée pour l'implémentation d'un quantum.

Travail demandé :

Etudier l'interruption 1CH et son implémentation ensuite écrire les deux programmes suivants et vérifier leur bon fonctionnement en effectuant les tests nécessaires.

Programme 1 :

Ecrire un programme **MODULAIRE** dénommé 1_tache permettant, à l'aide de l'interruption 1CH, de provoquer la suspension du programme principal en cours d'exécution et l'exécution **toutes les secondes** d'une procédure qui affichera le message suivant '1 sec écoulée

Le programme principal affiche sur l'écran le message répété 'Début du quantum de temps logiciel'. Le délai entre deux messages consécutifs est fixé par 2 boucles imbriquées insérées dans le programme principal.

Une fois compilé, le programme 1_tache devra donner l'écran suivant :

```
deroutement fait....
```

```
**** Début du quantum de temps logiciel **** 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée.....
**** Début du quantum de temps logiciel **** 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée.....
**** Début du quantum de temps logiciel **** 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1
sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée.....
1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée..... 1 sec écoulée.....
```

Le programme 1_tache continuera de tourner sans s'arrêter, et le message '1 sec écoulée....' sera affiché chaque seconde.

Programme 2 :

Ecrire un autre programme **MODULAIRE** dénommé 4_taches, permettant à l'aide de l'interruption périodique 1CH d'interrompre le programme principal en cours d'exécution, de lancer toutes les cinq secondes une tache parmi 4 taches dans l'ordre suivant : tache1, tache2, tache3, tache4.

Chaque tache se charge seulement d'afficher son propre message :

Message de la tache 1 : 'Tache 1 est en cours d'exécution....'

Message de la tache 2 : 'Tache 2 est en cours d'exécution....'

Message de la tache 3 : 'Tache 3 est en cours d'exécution....'

Message de la tache 4 : 'Tache 4 est en cours d'exécution....'

Une fois compilé, le programme 4_taches devra donner l'écran suivant :

```
deroutement fait....
```

```
tache1 en cours d"execution....
tache2 en cours d"execution....
tache3 en cours d"execution....
tache4 en cours d"execution....
```

```
tache1 en cours d"execution....
tache2 en cours d"execution....
tache3 en cours d"execution....
tache4 en cours d"execution....
```

```
tache1 en cours d"execution....
tache2 en cours d"execution....
tache3 en cours d"execution....
tache4 en cours d"execution....
```

Le programme 4_taches continuera de tourner sans s'arrêter et exécutera une tache chaque 5 secondes.

OUTIL UTILISÉ :

GUI Turbo Assembler x64 qu'il faudrait télécharger et installer.

TRAVAIL A REALISER:

1. Remettre vos deux programmes sources *1_tache.asm* et *4_taches.asm* optimisés, bien commentés et écrits sous forme procédurale, aucune redondance du code ne sera tolérée.
2. Une capture d'écran pour chaque programme montrant les résultats d'exécution obtenus.
3. Modifier les deux programmes pour que chacun deux s'arrête après 5 min d'exécution et refaire le travail demandé dans les questions (1) et (2).

MODALITE DE NOTATION:

Pour la notation de ce TP, il sera pris en considération la qualité de l'écriture du code, sa modularité et seulement les commentaires ayant un intérêt de clarification du code et bien entendu un travail personnel. ça sera le TP le plus noté.

REMARQUE: La note du TP sera revue à la baisse en cas d'absence ou de faible modularité du code, mais également si le code n'est pas optimisé ou non commenté.